چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۷

معماری پیشرفته — Plugin ،State Machine و Server-Driven UI و تکامل اپ‌های بزرگ



اسماعیل مرادی

@esmaeeil-moradi700

چرا معماری پیشرفته در ۲۰۲۶ مهم است؟

روز ۳ دربارهٔ یک صفحه و یک اپ بود. مصاحبه‌های آرکیتکت می‌پرسند وقتی **چند تیم، چند پلتفرم و سال‌ها انتشار** به آن معماری برسد چه می‌شود:

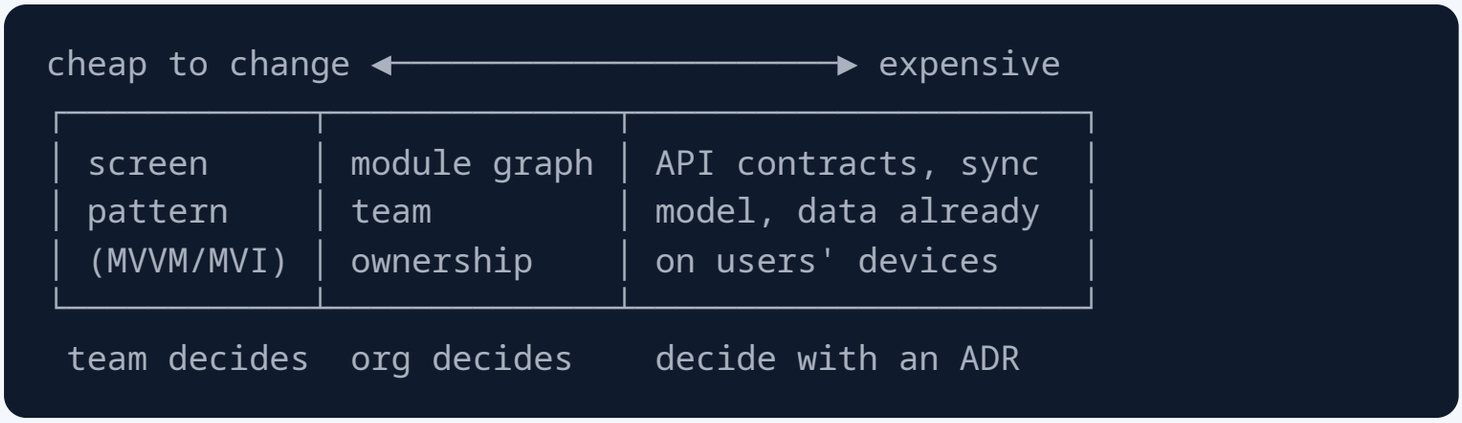
- چند تیم **یک باینری** منتشر می‌کنند ← مالکیت، قرارداد، pluginها
- کاربرها مدت‌ها **نسخه‌های قدیمی اپ** را اجرا می‌کنند ← APIهای سازگار با گذشته، server-driven UI، فلگ‌ها
- **flowهای پیچیده** (پرداخت، تماس، onboarding) ← state machineهای صریح
- **شبکهٔ بد و دادهٔ زنده** ← یک pipeline واحد offline-first
- تصمیم‌ها **بیشتر از تصمیم‌گیرنده‌ها عمر می‌کنند** ← ADR و راهبرد مهاجرت

سطح	سؤال‌ها
جونیور	چه چیزی در مقیاس بزرگ عوض می‌شود، مدل‌سازی domain، فرق event و command و state
مید	state machine، معماری plugin، دغدغه‌های مشترک، CQRS و event sourcing روی گوشی
سینیور	server-driven UI، تیم‌های فیچر و «micro-frontend»، آفلاین و real-time در سطح محصول
آرکیتکت	تصمیم‌گیری و ADR، تکامل معماری، اپ به‌عنوان بخشی از یک سیستم توزیع‌شده



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: معماری را جایی خرج کنید که تغییر گران است



- **معماری یعنی تصمیم‌هایی که برگرداندنشان گران است** — نه ساختار پوشه‌ها
- الگوی صفحه (روز ۳) ارزان عوض می‌شود؛ قرارداد با بک‌اند و داده‌ای که روی میلیون‌ها دستگاه نشسته، نه
- دقت را جایی خرج کنید که برگشت گران است؛ بقیه را ساده و قابل‌تغییر نگه دارید

سؤال یک آرکیتکت

«دو سال دیگر تغییر چه چیزی از همه سخت‌تر است — و امروز چطور آن گزینه را باز نگه داریم؟»



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

چه چیزی در مقیاس بزرگ عوض می‌شود، مدل‌سازی domain، فرق event و command و state



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. وقتی اپ بزرگ می‌شود چه چیزی عوض می‌شود

نیرو	جواب معماری
چند تیم، یک باینری	مالکیت ماژول، قرارداد `api:` (روز ۲۵)، pluginها (سؤال ۵)
کاربرها دیر آپدیت می‌کنند	APIهای سازگار با گذشته، server-driven UI، فلگ‌ها (سؤال ۸ و ۱۳)
flowهای پیچیدهٔ چندمرحله‌ای	state machineهای صریح (سؤال ۴)
شبکهٔ بد، دادهٔ زنده	یک pipeline واحد offline-first (سؤال ۱۰)
کدبیسی که سال‌ها زنده می‌ماند	ADR و مهاجرت تدریجی (سؤال ۱۱ و ۱۲)

قانون Conway: هر سیستم ساختار ارتباطی سازمانی را که آن را می‌سازد تقلید می‌کند. مرز ماژول‌ها آخر سر از مرز تیم‌ها پیروی می‌کند — پس هر دو را با هم طراحی کنید.

تلهٔ مصاحبه

«*معماری در مقیاس بزرگ یعنی لایه‌های بیشتر.*» — لایه، واسطه اضافه می‌کند نه مقیاس‌پذیری. چیزی که مقیاس می‌گیرد مالکیت روشن، قرارداد پایدار و تغییر مستقل است.

تلهٔ مصاحبه

«*در Clean Architecture، لایهٔ data داخلی است و لایهٔ presentation بیرونی‌ترین.*» (توصیف نسخهٔ پارسال) — UI و data **هر دو** adapterهای بیرونی‌اند که به domain وابسته‌اند (روز ۳).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. مدل‌سازی domain در Kotlin

```kotlin
@JvmInline
value class Money(val minor: Long) { // smallest currency unit
    init { require(minor >= 0) { "Negative amount" } }
    operator fun plus(other: Money) = Money(minor + other.minor)
}

@JvmInline value class OrderId(val raw: String)

data class Order(val id: OrderId, val lines: List<Line>) {
    init { require(lines.isNotEmpty()) { "Empty order" } }

    val total: Money
        get() = lines.fold(Money(0)) { sum, l -> sum + l.price }

    fun add(line: Line): Order = copy(lines = lines + line)
}
```

- **value object** (`Money` و `OrderId`): هویت ندارد و هنگام ساخت اعتبارسنجی می‌شود؛ یک `value class` معمولاً هزینهٔ allocation ندارد
- **entity و aggregate** (`Order`): هویت به‌علاوهٔ قواعد ثابت (invariant)؛ `copy()` هم از constructor رد می‌شود، پس چک‌های `init` باز هم اجرا می‌شوند
- یک `OrderId` را نمی‌شود جایی فرستاد که `UserId` انتظار می‌رود — کامپایلر جلویش را می‌گیرد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. bounded context و تله‌ها

bounded context: برای هر بخش از بیزینس یک مدل. «محصول» در کاتالوگ (توضیح، تصویر) همان «محصول» در سبد خرید (قیمت، تعداد) نیست. هر context به‌طور طبیعی به یک ماژول فیچر نگاشت می‌شود (روز ۲۵).

تلهٔ مصاحبه

«*پول یعنی* `Double`.» — ممیز شناور دودویی نمی‌تواند عدد ۰٫۱ را دقیق نشان دهد؛ جمع‌ها با خطای گرد کردن منحرف می‌شوند. کوچک‌ترین واحد پول را به شکل `Long` (یا `BigDecimal`) داخل یک value class نگه دارید.

تلهٔ مصاحبه

«*یک مدل* `Product` *مشترک در* `:core:model` *برای همهٔ فیچرها.*» — نیازهای همهٔ فیچرها به آن نشت می‌کند و هر تغییرش همهٔ ماژول‌ها را دوباره می‌سازد. هر context مالک مدل خودش است؛ در مرز تبدیل کنید.

سؤال تکمیلی

«*وقتی سرور اعتبارسنجی می‌کند، چرا در کلاینت هم؟*» — مرجع نهایی سرور است. invariantهای سمت کلاینت بازخورد فوری می‌دهند و حالت‌های غیرممکن را از UI بیرون نگه می‌دارند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. event، command و state

نوع	معنی	مثال	ابزار در Kotlin
command	دستور انجام یک کار — ممکن است رد شود، یک handler	`PlaceOrder`	یک `suspend fun`، یک ردیف outbox
event	گزارش اتفاقی که افتاده — یک واقعیت، صفر یا چند شنونده	`OrderPlaced`	یک `SharedFlow`، یک log ذخیره‌شده
state	آنچه همین حالا درست است — فقط آخرین مقدار مهم است	`cartCount = 3`	یک `StateFlow`، یک `Flow` از Room

```
// "User logged in" is state, not an event
interface SessionRepository {
    val session: StateFlow<Session>
}

// A late subscriber still gets the current value
sessionRepository.session
    .map { it.userId }
    .distinctUntilChanged()
    .onEach { userId -> analytics.setUserId(userId) }
    .launchIn(appScope)
```

تلهٔ مصاحبه

«رویداد `UserLoggedIn` را روی یک event bus سراسری با `SharedFlow` منتشر کن.» (مثال نسخهٔ پارسال) — با `replay = 0`، subscriberی که دیر شروع کند — مثلاً analytics که بعد از لاگین مقداردهی شده — هیچ‌وقت آن را نمی‌بیند. لاگین **state** است: آن را از مالکش منتشر کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

state machine، معماری plugin، دغدغه‌های مشترک، CQRS و event sourcing روی گوشی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. state machine برای flowهای پیچیده

```
sealed interface PayState {
    data object Idle : PayState
    data class Paying(val orderId: String) : PayState
    data class Paid(val receipt: String) : PayState
    data class Failed(val orderId: String) : PayState
}
sealed interface PayEvent {
    data class Pay(val orderId: String) : PayEvent
    data class Approved(val receipt: String) : PayEvent
    data object Declined : PayEvent
    data object Retry : PayEvent
}

fun PayState.on(e: PayEvent): PayState = when {
    this is Idle && e is Pay -> Paying(e.orderId)
    this is Paying && e is Approved -> Paid(e.receipt)
    this is Paying && e == Declined -> Failed(orderId)
    this is Failed && e == Retry -> Paying(orderId)
    else -> this // not allowed in this state: ignore
}
```

حالت‌های صریح، **گذارهای مجاز** صریح، و رد کردن بقیه. دو بار زدن دکمهٔ «پرداخت» در حالت `Paying` هیچ اثری ندارد.

تلهٔ مصاحبه

«*reducer در MVI خودش یک state machine است.*» — فقط اگر eventهایی را که در حالت فعلی نامعتبرند رد کند. reducerی که هر intent را در هر حالتی می‌پذیرد فقط یک `when` بزرگ است؛ ارزش state machine در گذارهایی است که **ممنوع** می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. اجرای state machine

```
class PaymentViewModel(
    private val api: PaymentApi,
) : ViewModel() {
    private val _state = MutableStateFlow<PayState>(Idle)
    val state: StateFlow<PayState> = _state.asStateFlow()

    fun send(e: PayEvent) { // called on the main thread
        val next = _state.value.on(e)
        if (next == _state.value) return
        _state.value = next
        if (next is Paying) authorize(next.orderId)
    }

    private fun authorize(id: String) = viewModelScope.launch {
        when (val r = api.authorize(id)) { // sealed result
            is AuthResult.Ok -> send(Approved(r.receipt))
            AuthResult.Declined -> send(Declined)
        }
    }
}
```

- side effectها **هنگام ورود به یک حالت** شروع می‌شوند و نتیجه‌شان به شکل event برمی‌گردد؛ تابع `on()` خالص می‌ماند و هر یال بدون mock تست می‌شود

سؤال تکمیلی

«*اگر وسط پرداخت process از بین برود؟*» — حالت و شناسهٔ سفارش را در `SavedStateHandle` یا Room ذخیره کنید؛ بعد از شروع دوباره وضعیت را از سرور بپرسید — هرگز کورکورانه دوباره «پرداخت» نفرستید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. معماری plugin: نقطه‌های توسعه

```
// :core:home:api — the extension point
interface HomeSection {
    val key: String
    val order: Int
    @Composable fun Content(modifier: Modifier)
}

// :feature:loyalty:impl — one plugin
class LoyaltySection @Inject constructor() : HomeSection {
    override val key = "loyalty"
    override val order = 20
    @Composable override fun Content(modifier: Modifier) {
        LoyaltyCard(modifier)
    }
}

@Module @InstallIn(SingletonComponent::class)
interface LoyaltyModule {
    @Binds @IntoSet fun section(s: LoyaltySection): HomeSection
}
```

میزبان interface را تعریف می‌کند؛ فیچرها از طریق DI multibinding (با `@IntoSet` یا `@IntoMap`) به آن وصل می‌شوند. میزبان هیچ‌وقت یک فیچر را import نمی‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. سمت میزبان

```
// :feature:home:impl — knows only the interface
class HomeViewModel @Inject constructor(
    all: Set<@JvmSuppressWildcards HomeSection>,
    flags: FeatureFlags,
) : ViewModel() {
    val sections: List<HomeSection> = all
        .filter { flags.isEnabled(it.key) }
        .sortedBy { it.order }
}
```

- نقطه‌های توسعهٔ رایج: بخش‌های صفحهٔ خانه، آیتم‌های تنظیمات، handlerهای deep link، کارهای startup، مقصدهای analytics، روش‌های پرداخت
- اضافه کردن فیچر یعنی اضافه کردن یک ماژول؛ حذفش یعنی پاک کردن یک خط وابستگی در `app`:

تلهٔ مصاحبه

«pluginها به همان ترتیبی اجرا می‌شوند که ماژول‌هایشان اضافه شده‌اند.» — یک `Set` از multibinding هیچ قراردادی دربارهٔ ترتیب ندارد. با یک کلید صریح مرتب کنید.

تلهٔ مصاحبه

«UI هر plugin را در try/catch بپیچ تا یک plugin خراب صفحهٔ خانه را crash نکند.» — Compose اجازهٔ try/catch دور فراخوانی composableها را نمی‌دهد. دادهٔ plugin را در ViewModel اعتبارسنجی کنید، UI آن را ساده نگه دارید و pluginهای پرریسک را پشت یک فلگ ریموت بگذارید.

اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. دغدغه‌های مشترک (cross-cutting)

دغدغه	جایش کجاست
هدرهای احراز هویت، لاگ، retry	interceptorهای HTTP یا pluginهای Ktor (روز ۱۱)
زمان‌سنجی و cache دور یک repository	یک decorator که با DI وصل می‌شود
analytics صفحه‌ها	یک listener روی ناوبری — نه کد در تک‌تک صفحه‌ها
لاگ و context برای crash	یک interface به اسم `Logger` با مقصدهای قابل‌تعویض
فلگ‌ها و remote config	یک interface تزریق‌شده به اسم `FeatureFlags`

تلهٔ مصاحبه

«لاگ، analytics و مدیریت خطا را در یک `BaseActivity` یا `BaseViewModel` بگذار.» — ارث‌بری دغدغه‌های مشترک را به یک کلاس پایهٔ غول‌پیکر تبدیل می‌کند: همهٔ صفحه‌ها باید از آن ارث ببرند، هیچ‌کس نمی‌تواند کنار بکشد، و فقط یک superclass دارید. به‌جایش رفتارها را ترکیب کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. decorator به‌جای ارث‌بری

```
class TimedFeedRepository(
    private val delegate: FeedRepository,
    private val metrics: Metrics,
) : FeedRepository by delegate {
    override suspend fun refresh() {
        val mark = TimeSource.Monotonic.markNow()
        try {
            delegate.refresh()
        } finally {
            metrics.record("feed_refresh", mark.elapsedNow())
        }
    }
}
```

- عبارت `by delegate` بقیهٔ متدها را مستقیم پاس می‌دهد؛ فقط `refresh()` پیچیده می‌شود
- DI تصمیم می‌گیرد اپ نسخهٔ ساده را بگیرد یا نسخهٔ زمان‌سنجی‌شده را — صدازننده‌ها فرقی نمی‌بینند

تلهٔ مصاحبه

«*برای لاگ از AOP با AspectJ استفاده کن.*» (توصیهٔ نسخهٔ پارسال) — bytecode weaving در اندروید یعنی pluginهای build شکننده و رفتاری که هیچ‌کس در کد نمی‌بیند. decorator و interceptor همین کار را صریح انجام می‌دهند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. CQRS و event sourcing — روی گوشی؟

الگو	روی سرور	روی دستگاه
CQRS: مدل خواندن و نوشتن جدا	خواندن مستقل مقیاس می‌گیرد	همین حالا دارید: خواندن‌ها projectionهای Room به شکل `Flow` اند، نوشتن‌ها commandهایی داخل transaction
event sourcing: یعنی state از بازپخش eventها ساخته شود	ردپای ممیزی، دفتر کل	به‌ندرت: منبع حقیقت سرور است؛ یک log محلی بی‌انتها فضا و زمان startup می‌گیرد
outbox: صف کردن commandها و ارسال بعدی	پیام‌رسانی مطمئن	بله — نوشتن در حالت آفلاین (سؤال ۱۰)

```
data class OrderRow(val id: String, val title: String)

@Dao
interface OrderDao { // a read model shaped for one screen
    @Query("SELECT id, title FROM orders")
    fun observeRows(): Flow<List<OrderRow>>
}
```

تلهٔ مصاحبه

«موجودی حساب را روی دستگاه event-source کن: هر واریز و برداشت را ذخیره کن و برای محاسبه‌اش بازپخش کن.» (طراحی نسخهٔ پارسال) — دفتر کل روی سرور است. اپ موجودی سرور را cache می‌کند و commandهای خودش را در صف می‌گذارد.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

server-driven UI، تیم‌های فیچر و «micro-frontend»، آفلاین و real-time در سطح محصول



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. server-driven UI

سرور می‌فرستد **چه چیزی** نشان داده شود — فهرستی از کامپوننت‌هایی که اپ از قبل می‌شناسد. اپ تصمیم می‌گیرد **چطور** رندرشان کند.

```
@Serializable
sealed interface Block {
    @Serializable @SerialName("header")
    data class Header(val title: String) : Block

    @Serializable @SerialName("carousel")
    data class Carousel(val ids: List<String>) : Block
}

@Composable
fun BlockView(block: Block, onOpen: (String) -> Unit) {
    when (block) {
        is Block.Header -> SectionHeader(block.title)
        is Block.Carousel -> ProductCarousel(block.ids, onOpen)
    }
}
```

برد	هزینه
تغییر چیدمان و ترتیب بدون انتشار اپ	قراردادی که باید با همهٔ نسخه‌های فعال اپ سازگار بماند
آزمایش سمت سرور، یک چیدمان برای اندروید، iOS و وب	تست، preview و بررسی دسترس‌پذیری سخت‌تر؛ cache آخرین چیدمان برای حالت آفلاین



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. زنده ماندن با blockهای ناشناخته

```
val json = Json { ignoreUnknownKeys = true }

// One unknown block must not break the whole screen
fun decodeBlocks(body: String): List<Block> =
    json.parseToJsonElement(body).jsonArray.mapNotNull { el ->
        try {
            json.decodeFromJsonElement<Block>(el)
        } catch (e: SerializationException) {
            null // a block type this app version doesn't know
        }
    }
```

- به‌طور پیش‌فرض، یک `type` ناشناخته باعث می‌شود decode **کل** فهرست خطا بدهد — عنصر به عنصر decode کنید
- نسخهٔ اپ را همراه درخواست بفرستید تا سرور فقط blockهایی را بفرستد که آن نسخه پشتیبانی می‌کند

تلهٔ مصاحبه

«با SDUI دیگر هیچ‌وقت به انتشار اپ نیاز نداریم.» — فقط برای چیدمان‌هایی که از کامپوننت‌های شناخته‌شده برای اپ نصب‌شده ساخته شده‌اند. کامپوننت جدید یعنی انتشار جدید. و Google Play دانلود کد اجرایی (dex، کتابخانهٔ native) از بیرون Play را ممنوع کرده است — چیدمان سمت سرور داده است، نه کد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تیم‌های فیچر و «micro-frontend»

تیم پلتفرم مالک است	تیم‌های فیچر مالک‌اند
build، CI، convention pluginها	ماژول‌های فیچر خودشان، هم `:api` و هم `:impl`
design system و ماژول‌های core	صفحه‌ها، ViewModelها و دادهٔ domain خودشان
قطار انتشار و داشبوردهای سلامت اپ	فلگ‌ها، متریک‌ها و on-call فیچر خودشان
گاردریل‌ها: قواعد گراف، Lint، Konsist	تصمیم‌های داخل ماژول‌هایشان (MVVM یا MVI)

- **مانور معکوس Conway:** تیم‌ها را همان شکلی بچینید که می‌خواهید معماری شکل بگیرد
- **یک باینری، یک قطار انتشار:** فیچرها پشت فلگ merge می‌شوند و با آهنگی ثابت منتشر می‌شوند — deploy و release از هم جدا هستند (روز ۲۴)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. «مستقل» در یک APK یعنی چه

تلهٔ مصاحبه

«*micro-frontend به هر تیم اجازه می‌دهد با چرخهٔ انتشار و تکنولوژی خودش منتشر کند.*» (ادعای نسخهٔ پارسال) — داخل یک APK چنین چیزی ممکن نیست. یک باینری، یک process، یک گراف وابستگی (نسخه‌های متضاد یک کتابخانه به یک نسخه می‌رسند) و یک انتشار وجود دارد. استقلال از مرز ماژول‌ها و فلگ‌ها می‌آید.

سؤال تکمیلی

«*چطور نگذاریم تیم پلتفرم گلوگاه شود؟*» — مدل inner source: تیم‌های فیچر با pull request در ماژول‌های core مشارکت می‌کنند و تیم پلتفرم مالک review و کیفیت است. به‌جای گرفتن تأیید، «جادهٔ آسفالته» بسازید: template و convention plugin.

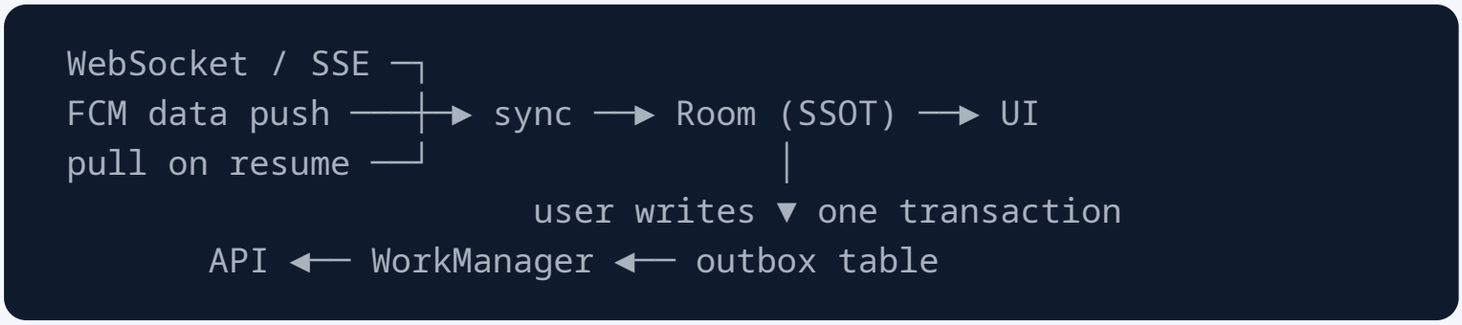
نشانه‌های ساختار تیمی سالم

یک فیچر معمولی فقط ماژول‌های خودش را تغییر می‌دهد. تیم‌ها بدون هماهنگی پشت فلگ منتشر می‌کنند. با اضافه شدن تیم‌ها، زمان build و review ثابت می‌ماند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. آفلاین و real-time در سطح محصول



رفتار آفلاین یک **تصمیم محصولی برای هر فیچر** است، نه یک تنظیم کتابخانه:

فیچر	خواندن آفلاین	نوشتن آفلاین	تعارض
چت	تاریخچهٔ cacheشده	در صف، با برچسب «در حال ارسال»	فقط افزودنی؛ ترتیب پیام‌ها را سرور تعیین می‌کند
یادداشت	بله	بله	ادغام فیلدبه‌فیلد؛ OT یا CRDT برای متن مشترک
پرداخت	آخرین وضعیت معلوم	**نه** — فقط آنلاین	سرور تصمیم می‌گیرد
فید	صفحه‌های cacheشده	—	حرف آخر با سرور است



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. یک pipeline برای همه‌چیز

```
@Dao
abstract class MessageDao {
    @Insert abstract suspend fun insert(m: MessageEntity)
    @Insert abstract suspend fun enqueue(op: OutboxEntry)

    // Optimistic write + pending command, atomically
    @Transaction
    open suspend fun send(m: MessageEntity, op: OutboxEntry) {
        insert(m.copy(status = Status.SENDING))
        enqueue(op)
    }
}
```

- کانال‌های real-time فقط **دیتابیس را پر می‌کنند**؛ UI مثل همهٔ داده‌های دیگر Room را observe می‌کند
- یک unique work در WorkManager با شرط شبکه، outbox را خالی می‌کند (روز ۱۴ و ۱۵)؛ هر retry یک idempotency key همراه دارد (سؤال ۱۳)

تلهٔ مصاحبه

«*برای real-time، UI مستقیم به socket گوش بدهد.*» — حالا دو منبع حقیقت دارید، و آفلاین، اتصال دوباره و process death هر کدام مسیر کد جداگانه می‌خواهند.

تلهٔ مصاحبه

«*آخرین نوشتن برنده است (LWW) پیش‌فرض ساده‌ای است.*» — با ساعت دستگاه‌ها، اختلاف ساعت باعث می‌شود یک ویرایش قدیمی‌تر برنده شود. نسخه‌هایی را مقایسه کنید که سرور تعیین کرده، و سیاست را برای هر فیچر جدا انتخاب کنید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

تصمیم‌گیری و ADR، تکامل معماری، اپ به‌عنوان بخشی از یک سیستم توزیع‌شده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. سند تصمیم معماری (ADR)

```
# ADR-014: Server-driven layout for the home screen
Status: Accepted · Supersedes ADR-009
Context: Home changes every sprint; many users run
  old app versions; three teams edit home.
Decision: Home is a list of server blocks rendered
  by a client registry. Unknown blocks are skipped.
Options rejected: WebView home; weekly releases.
Consequences:
  + Merchandising ships without an app release
  - Versioned contract, tests on both sides
  - New block types still need an app release
Revisit: if home latency or crash rate regresses
```

- کوتاه، داخل ریپو و کنار کد، و reviewشده مثل کد
- **تغییرناپذیر:** وقتی تصمیمی عوض می‌شود، یک ADR جدید جایگزین قبلی می‌شود
- گزینه‌های ردشده و *چرایی* تصمیم، همان چیزی است که تیم‌های آینده بیشتر از همه لازم دارند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. تصمیم‌ها چطور گرفته می‌شوند

تصمیم	فرایند
برگشت‌پذیر و محلی — الگوی یک صفحه، یک کتابخانه داخل یک ماژول	تیم تصمیم می‌گیرد؛ بدون تشریفات
سخت‌برگشت یا بین‌تیمی — مدل همگام‌سازی، قرارداد API، فریم‌ورک DI، قواعد ماژول	پیشنهاد مکتوب، یک spike، یک review تیم‌های درگیر، و یک ADR
نامطمئن	یک نمونهٔ اولیهٔ زمان‌دار با معیار موفقیت، قبل از تعهد

تلهٔ مصاحبه

«*یک کمیتهٔ معماری همهٔ پیشنهادها را تأیید می‌کند.*» (چارچوب نسخهٔ پارسال) — کمیته گلوگاه می‌شود و تیم‌ها دورش می‌زنند. نزدیک به مسئله تصمیم بگیرید؛ فقط تصمیم‌های برگشت‌ناپذیر و بین‌تیمی را بالا ببرید — و مکتوبشان کنید.

سؤال تکمیلی

«*گزینه‌ها را چطور مقایسه می‌کنید؟*» — کیفیت‌هایی را که این‌جا مهم‌اند نام ببرید (سرعت انتشار، آفلاین، زمان build، استقلال تیم‌ها)، هر گزینه را با آن‌ها بسنجید، و پرریسک‌ترین فرض را اول با نمونهٔ اولیه امتحان کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. تکامل یک معماری

راهبرد	چطور	مثال
Strangler fig	کد جدید با سبک جدید؛ بخش‌های قدیمی یکی‌یکی جایگزین می‌شوند تا چیزی نماند	از Views به Compose، صفحه به صفحه
Branch by abstraction	یک interface جلوی کد قدیمی؛ نسخهٔ جدید پشتش ساخته می‌شود؛ سوییچ	از پشتهٔ شبکهٔ قدیمی به Ktor
Parallel run	هر دو را اجرا کن، نتیجه‌ها را مقایسه کن، همچنان از قدیمی استفاده کن	منطق جدید قیمت‌گذاری یا همگام‌سازی
انتشار با فلگ	سوییچ برای هر گروه کاربر، پایش متریک‌ها، برگشت فوری	هر جایگزینی پرریسک

```
class SwitchingFeedRepository @Inject constructor(
    private val legacy: LegacyFeedRepository,
    private val modern: OfflineFirstFeedRepository,
    private val flags: FeatureFlags,
) : FeedRepository {
    override fun observeFeed(): Flow<List<Post>> =
        if (flags.isEnabled(Flag.NEW_FEED)) modern.observeFeed()
        else legacy.observeFeed()
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. برای مهاجرت پایان تعیین کنید

- **اول استفادهٔ جدید را متوقف کنید:** API قدیمی را با یک چک سفارشی Lint، قاعدهٔ `ForbiddenImport` در detekt یا یک تست Konsist ممنوع کنید — وگرنه مهاجرت هیچ‌وقت تمام نمی‌شود
- **پیشرفت را قابل‌دیدن کنید:** درصد مهاجرت هر ماژول، روی یک داشبورد
- **تمام شدن یعنی حذف شدن:** مسیر قدیمی، سوییچ و فلگ پاک می‌شوند

تلهٔ مصاحبه

«*بیایید اپ را از صفر بازنویسی کنیم.*» — کار فیچرها متوقف می‌شود در حالی که اپ قدیمی همچنان تغییر می‌کند، و بازنویسی دنبال هدفی متحرک می‌دود. مهاجرت تدریجی به انتشار ادامه می‌دهد و در هر قدم قابل‌برگشت است.

سؤال تکمیلی

«*دو الگو در یک کدبیس — یعنی ناهماهنگی؟*» — موقتاً، بله. به شرطی قابل‌قبول است که هدف در یک ADR مستند باشد، کد جدید از آن پیروی کند، و الگوی قدیمی برنامهٔ حذف داشته باشد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. اپ به‌عنوان بخشی از یک سیستم توزیع‌شده

- **نسخه‌های قدیمی زنده می‌مانند:** هر تغییر API باید با گذشته سازگار بماند — فیلدهای افزودنی، endpointهای نسخه‌دار، و حداقل نسخهٔ پشتیبانی‌شده که با in-app update اعمال می‌شود (روز ۲۴)
- **خوانندهٔ مداراگر (tolerant reader):** کلاینت فیلدهای ناشناخته را نادیده می‌گیرد و مقدارهای ناشناختهٔ enum را به یک مقدار پیش‌فرض نگاشت می‌کند
- **الگوی BFF (Backend for Frontend):** یک API هم‌شکل با صفحه‌های اپ که سرویس‌های بک‌اند را تجمیع می‌کند — رفت‌وبرگشت کمتر و نسخه‌بندی مخصوص اپ. GraphQL جواب دیگری است (روز ۱۲)
- **سازگاری نهایی (eventual consistency) یک حالت UI است:** «در انتظار»، «در حال همگام‌سازی»، «ناموفق — تلاش دوباره»

```
val json = Json {
    ignoreUnknownKeys = true // new fields from newer servers
    coerceInputValues = true // unknown enum value → default
}

@Serializable
data class OrderDto(
    val id: String,
    val status: OrderStatus = OrderStatus.UNKNOWN,
)
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. retry بدون تکرار

```kotlin
@Entity(tableName = "outbox")
data class OutboxEntry(
    @PrimaryKey val key: String = UUID.randomUUID().toString(),
    val path: String,
    val body: String,
)

// Every retry of the same command sends the same key
suspend fun HttpClient.send(entry: OutboxEntry) =
    post(entry.path) {
        header("Idempotency-Key", entry.key)
        contentType(ContentType.Application.Json)
        setBody(entry.body)
    }
```

تلهٔ مصاحبه

«*retry کردن یک POST ناموفق بی‌خطر است.*» — timeout به شما نمی‌گوید سرور درخواست را پردازش کرده یا نه. بدون idempotency key یک retry می‌تواند دو بار پول کم کند؛ با آن، سرور تکرار را تشخیص می‌دهد و همان نتیجهٔ اول را برمی‌گرداند.

جواب سینیور در یک جمله

«معماری در مقیاس بزرگ دربارهٔ تغییر است: تصمیم‌های گران را آگاهانه بگیر و مکتوب کن، قراردادها را با گذشته سازگار نگه دار، و برای هر مهاجرت پایانی تعیین کن.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۷ — خلاصهٔ یک‌صفحه‌ای

- معماری یعنی تصمیم‌هایی که برگرداندنشان گران است. دقت را همان‌جا خرج کنید.
- قانون Conway: مرز تیم‌ها و مرز ماژول‌ها همدیگر را شکل می‌دهند — هر دو را طراحی کنید.
- domain را با value class و invariant مدل کنید؛ هر bounded context مالک مدل خودش است. پول هرگز `Double` نیست.
- command درخواست می‌کند، event گزارش می‌دهد، state توصیف می‌کند — «لاگین شده» یک state است.
- state machine: حالت‌ها و گذارهای مجاز صریح — ارزشش در چیزهایی است که ممنوع می‌کند.
- plugin یعنی نقطهٔ توسعه به‌علاوهٔ DI multibinding؛ مرتب‌سازی صریح؛ بدون try/catch دور composableها.
- دغدغه‌های مشترک: decorator و interceptor — نه کلاس پایه و نه AspectJ.
- در SDUI، blockهای ناشناخته رد می‌شوند؛ کامپوننت جدید هنوز انتشار می‌خواهد.
- آفلاین و real-time: همهٔ منبع‌ها در دیتابیس می‌نویسند؛ نوشتن‌ها از یک outbox با idempotency key می‌گذرند.
- با ADR تصمیم بگیرید، با strangler fig و فلگ مهاجرت کنید، APIها را با نسخه‌های قدیمی سازگار نگه دارید.

روز ۲۸ در راه است: Performance در مقیاس بزرگ. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700